Module 11: Run Playbooks with Automation Controller

DURATION: 3 HOURS | THEORY: 25% | HANDS-ON: 75%

This module is where **automation becomes operational**. After completing this module, participants will be able to launch automation jobs, pass runtime parameters, schedule jobs, trigger automation via APIs and webhooks, monitor execution in real time, and work with job results and artifacts.

Built So Far

Inventories, projects, playbooks, and job templates are ready.

What's Next

Execute automation at scale using Automation Controller.

11.1 Launching Jobs in AAP

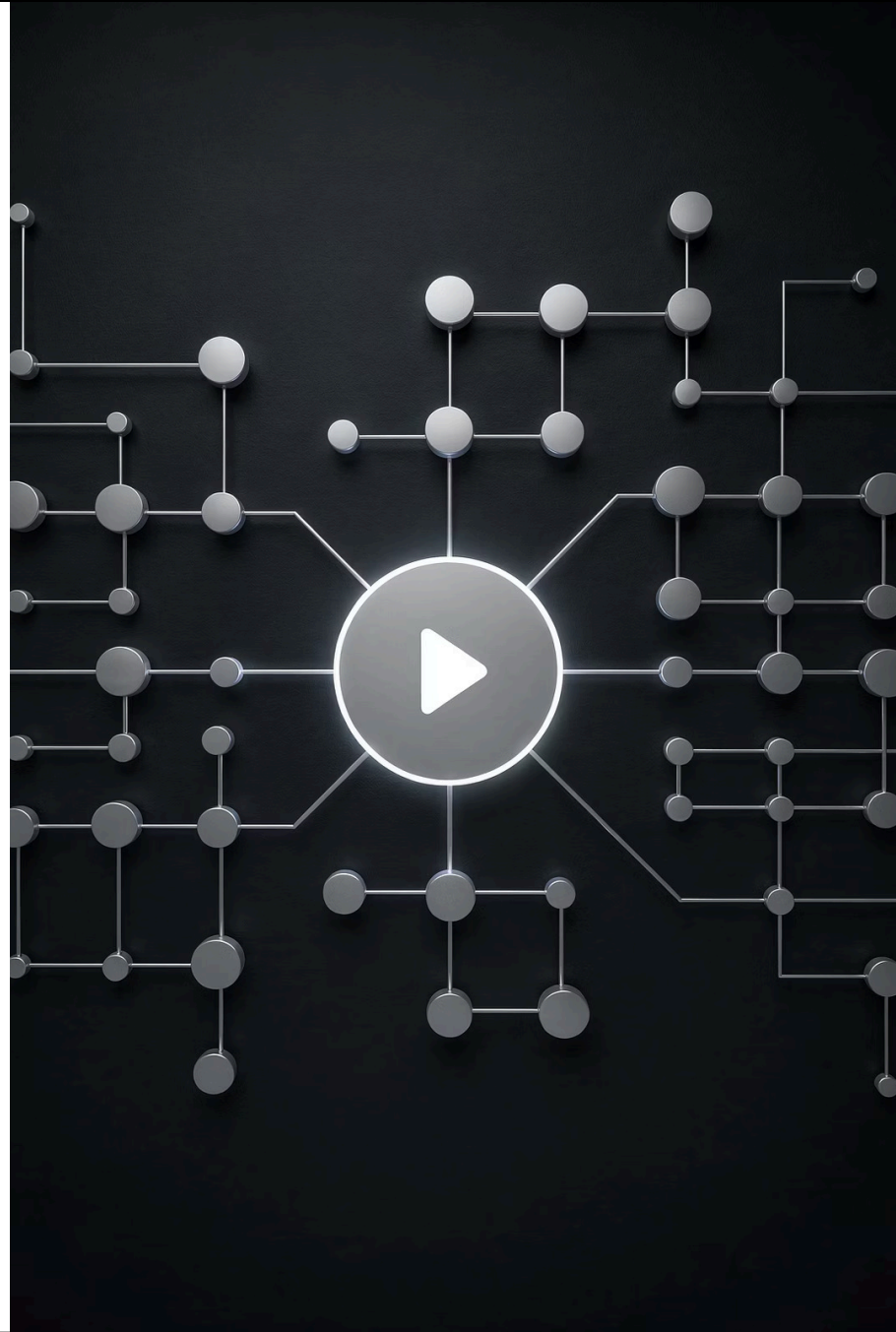
Automation Controller runs playbooks as **Jobs**. A **Job** is a runtime execution of a **Job Template** — the bridge between your defined automation settings and real-world execution across your infrastructure.

Job Template

Defines the automation settings — inventory, project, playbook, credentials, and variables.

Job Instance

Created at runtime when a user clicks **Launch**, executing the template against live infrastructure.



Manual Job Launches from the UI

Job Template Defines

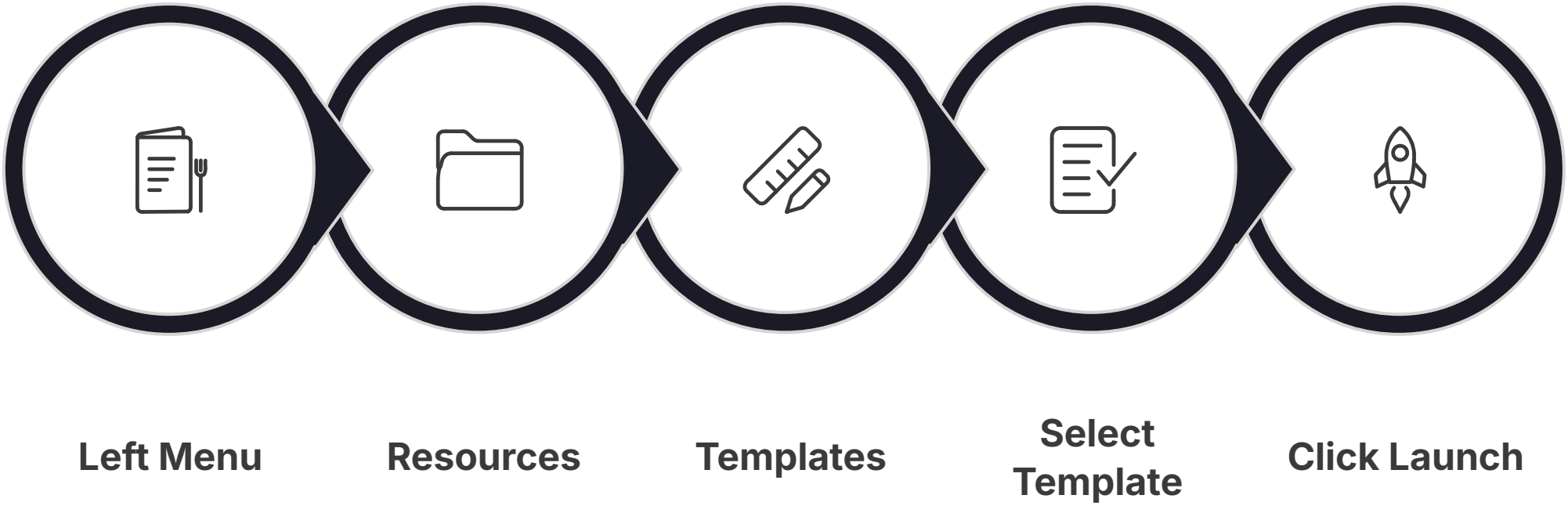
- Inventory
- Project
- Playbook
- Credentials
- Variables

How It Works

A **Job Template** stores all automation settings in one reusable configuration. When a user clicks **Launch**, a job instance is created from that template and executed against the defined inventory with the specified credentials and variables.

UI Navigation — AAP 2.4

Launching a job from the Automation Controller interface is straightforward. Follow the navigation path below to reach your Job Templates and initiate execution.



This path is consistent across AAP 2.4 and provides a centralized location for managing and executing all automation job templates.

Job Execution Flow

When you launch a job, Automation Controller orchestrates a precise sequence of steps to ensure secure, isolated, and reliable playbook execution.

01

Select Execution Environment

Controller identifies the appropriate container image.

02

Create Isolated Container

A temporary, sandboxed container is spun up for the job.

03

Pull Playbook & Load Inventory

Playbook is fetched from the project repo; inventory is loaded.

04

Inject Credentials & Execute

Credentials are securely injected; playbook runs via **ansible-runner**.

Internal Architecture

ansible-runner

The core engine that executes playbooks inside execution environments.

Execution Environments

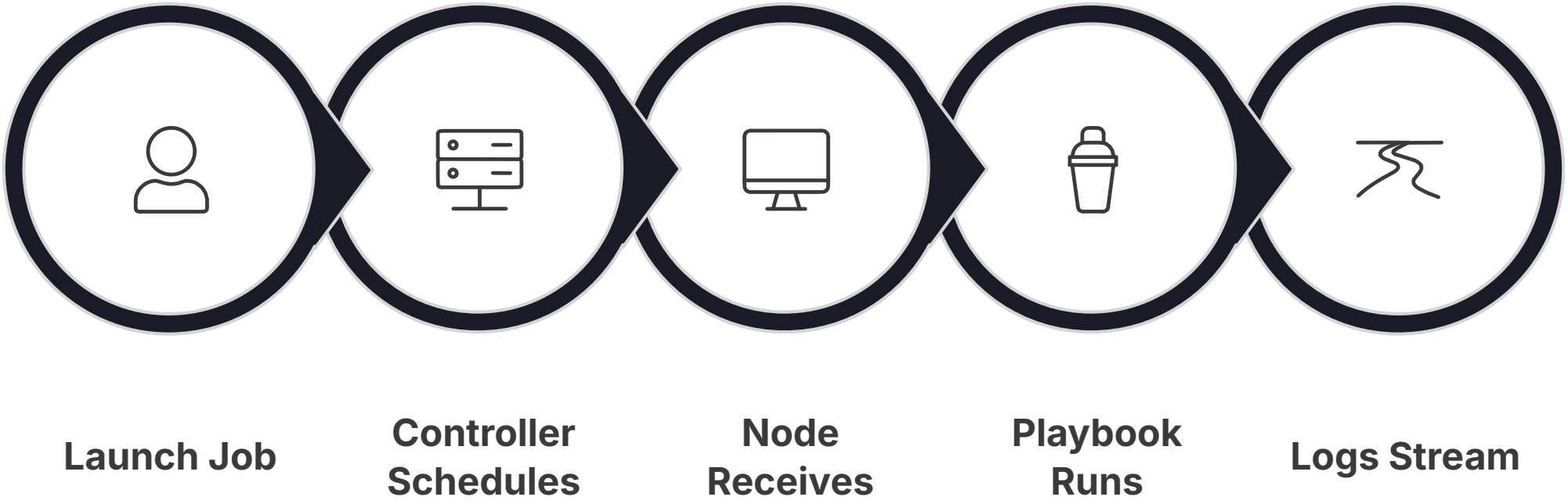
Containerized runtimes that provide consistent, isolated execution contexts.

Receptor Mesh

Execution occurs on **execution nodes**, not necessarily the controller itself.

Example: Launching install_nginx.yml

Here's how a typical job launch flows through the Automation Controller architecture — from user action to playbook completion.



Every step is logged and visible in real time, giving operators full visibility into execution progress and outcomes.

Enterprise Example: Deploy Web Application

200

Servers Targeted

Simultaneous deployment across the entire **webservers** inventory group.

1

Job Launch

A single click by a DevOps engineer triggers the full deployment pipeline.

A DevOps engineer runs the **Deploy Web Application** job template targeting the **webservers** group, resulting in the application being deployed across 200 servers simultaneously — no manual intervention required.

Lab Activity: Launch Your First Job

In this hands-on exercise, students will experience the full job launch workflow inside Automation Controller.

1

Go to Templates

Navigate to Resources → Templates in the left menu.

2

Select Template

Choose the **Deploy App Job Template** from the list.

3

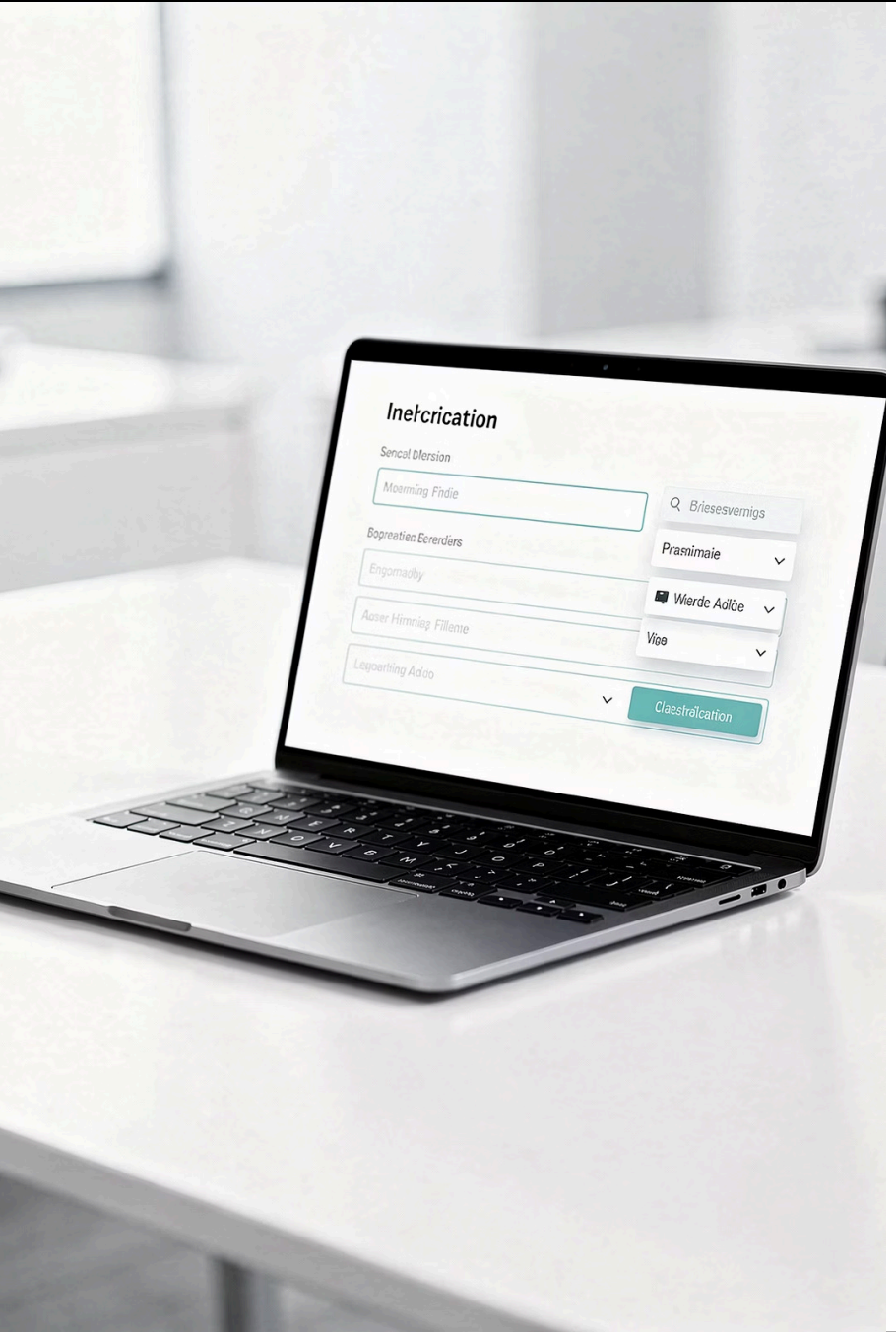
Click Launch

Initiate the job execution from the template detail view.

4

Watch Logs

Observe execution logs streaming in real time in the UI.



Providing Runtime Input via Surveys

Sometimes automation requires **user input** before execution. Rather than hardcoding values, AAP provides **Surveys** — dynamic forms that collect runtime variables such as target environment, version numbers, or feature flags.

dev

Development environment

qa

Quality assurance

prod

Production environment

What Is a Survey?

Definition

A **survey is a form** displayed to the user before a job is launched. It collects values that are passed as **variables into the playbook** at runtime.

Why It Matters

Surveys make job templates reusable and flexible. Instead of creating separate templates for each scenario, a single template can serve multiple use cases by accepting different inputs from different users or teams.

Example Survey Fields

Surveys support multiple question types to capture a wide range of runtime inputs from users before job execution.

Text Field

Question: Application Version
Captures free-form text input such as version numbers or identifiers.

Dropdown

Question: Environment
Presents a predefined list of options like dev, qa, or prod.

Yes/No

Question: Restart Service
Boolean input for enabling or disabling specific playbook behaviors.

Example Survey Variable in a Playbook

Survey responses are automatically mapped to playbook variables. Here's how the **Application Version** survey question flows into a playbook task:

Survey Question

Question: **Application Version**
Variable created: `app_version`

Playbook Usage

```
- name: Deploy application
  hosts: web
  vars:
  version: "{{ app_version }}"
```

Enterprise Example: Release Deployment Survey

A release engineer runs a deployment job. Before execution begins, the survey prompts for critical release parameters — ensuring the right version is deployed to the right environment with the correct rollback strategy.

Release Version

Specifies the exact build or tag to be deployed.

Environment

Selects the target environment: dev, qa, or prod.

Rollback Option

Enables automatic rollback if the deployment fails.

Lab Exercise: Build and Use a Survey

Students will configure a survey on an existing job template and experience how runtime input flows into playbook execution.

1 Edit Job Template

Open an existing template and navigate to the Survey tab.

2 Enable Survey & Add Fields

Toggle the survey on and add text, dropdown, and yes/no fields.

3 Launch & Enter Values

Launch the job, complete the survey form, and observe how variables are passed into the playbook.

Using Extra Variables at Launch

In addition to surveys, users can pass **extra variables manually** at launch time. This is ideal for ad-hoc overrides or scripted automation where a survey form isn't practical.

> Extra variables can be entered as **JSON or YAML** directly in the launch dialog — giving operators precise, on-demand control over playbook behavior without modifying the template.

```
{
 "app_version": "2.3",
 "environment": "prod"
}
```

Where to Add Extra Variables

When launching a job, the **Extra Variables** textbox accepts both JSON and YAML input. This field appears in the launch dialog and allows runtime overrides without modifying the job template.

JSON Format

```
{ "app_version": "2.3",
  "environment": "prod" }
```

YAML Format

```
app_version: 2.0
restart_service: true
```

Variable Precedence in Ansible

When multiple variable sources exist, Ansible follows a strict priority order. **Extra variables always win** — they override everything else, making them the most powerful (and potentially dangerous) input method.

1

Role Defaults

Lowest priority — easily overridden.

2

Inventory Variables

Host and group vars defined in inventory.

3

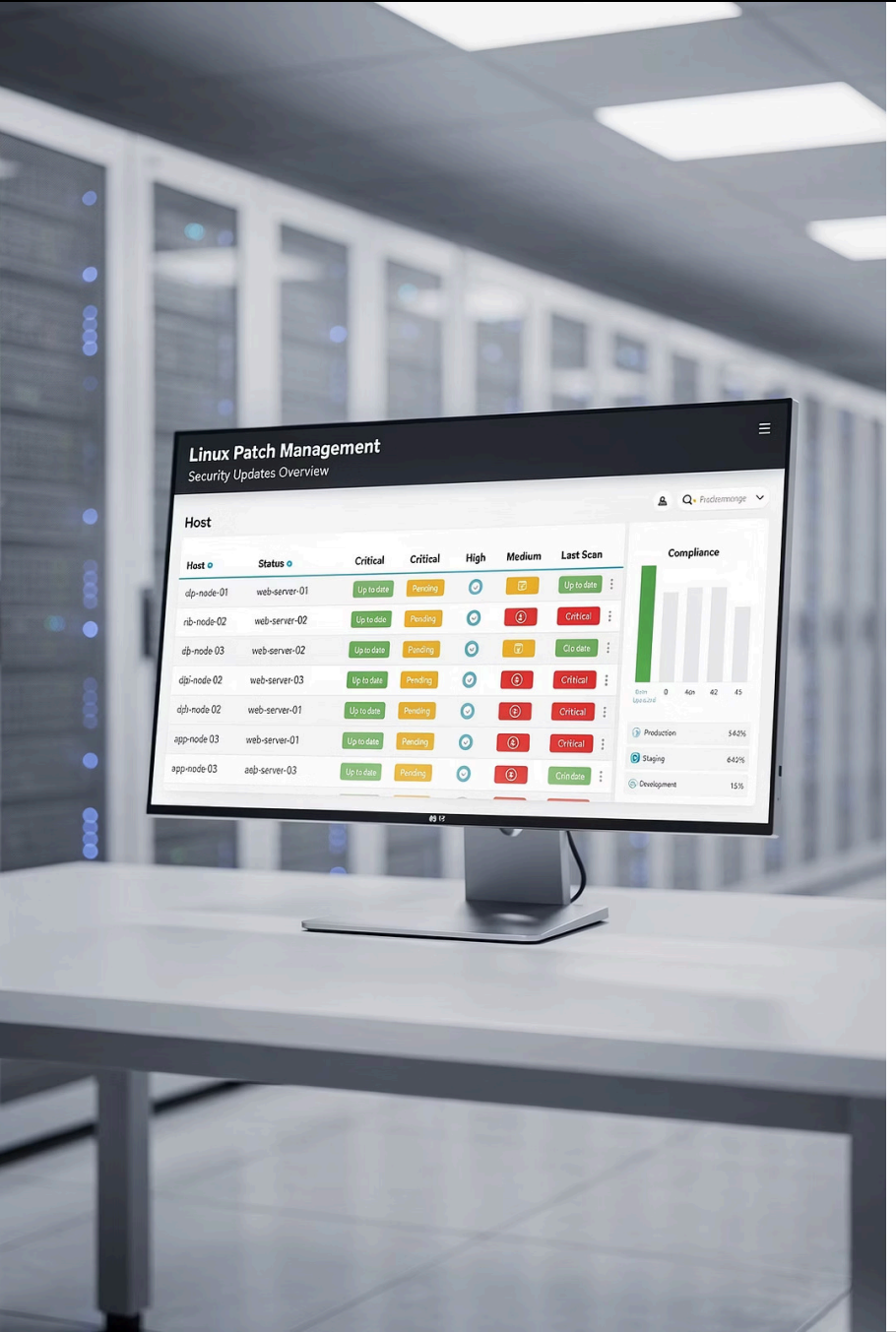
Survey Variables

Collected from the survey form at launch.

4

Extra Variables

Highest priority — overrides all other sources.



Example: Patch Linux Servers with Extra Variables

A DevOps engineer runs the **Patch Linux Servers** job template and needs to target only security-level patches — without modifying the template itself.

Job Template

Patch Linux Servers — a reusable template covering multiple patch levels.

Extra Variable Passed

`patch_level: security` — scopes execution to security patches only, overriding any default value.

Selecting Limits and Tags at Runtime

Two powerful runtime options give operators fine-grained control over **which hosts** are targeted and **which tasks** are executed — without modifying the job template.

Limits

Restrict job execution to specific hosts instead of the entire inventory. Example: target only `web01` and `web02` rather than all webservers.

Tags

Run only the playbook tasks marked with specific tags. Useful for partial deployments, targeted patches, or skipping certain steps during testing.